

NCF Waveform Pipeline — Real Correctness Verification + ObsPy Scaling Numbers + Next Task for wavenet_junior

Hi team (wavenet_junior, this one's mainly for you — everyone else, the ObsPy scaling numbers in End-notes feed directly into the AWS-vs-ObsPy cost report once the mothership scan lands),

Full technical detail lives in docs/ncf_pipeline_stages/ (PROGRESS.md is the entry point, hdf5_schema.md and HANDOFF_wavenet_junior.md have the code-level detail). This memo is structured in three parts: a TL;DR of where things stand, your actual tasks, and end-notes with the full evidence/context behind all of it — read the first two, use the third for follow-up.

TL;DR

■ ObsPy MassDownloader + Bluehive is the real, working pipeline choice, and our instrument-response removal is independently verified correct (correlation 0.9523 against ADAMA's own reference output) — not just non-crashing. Detail: End-notes A.

■ Decision (2026-09-24): download all available history for every station, not a bounded window — 78.9 TB total, up to 56 years for the oldest stations. Detail: End-notes B.

■ I launched the full 2,000-station run today and hit a real bug: ObsPy itself breaks on very wide date-range requests, sometimes crashing, sometimes silently reporting "no data" when there really is data. 0% of ~800 completed stations had actually succeeded before I caught it. Found, fixed (request one year at a time instead of all 56), and verified on the exact stations that broke. Detail: End-notes C.

■ While reviewing what else could go wrong, I found a second, still-unfixed gap: a station's processing currently has to complete in one uninterrupted pass, all in memory, or all of it is lost and its memory use for our biggest stations is untested. I'm testing the real numbers before building a fix, not guessing. Detail: End-notes D.

■ A 60-station canary batch is running right now, healthy, first real success already landed. This is what Task A below asks you to watch.

■ Bottom line for you: you now have an ongoing role watching live runs, not just the two one-off verification tasks from earlier this week. All three tasks are below.

YOUR TASKS

TASK A — Watch the live canary run (start this one today, it's active now)

I'm not asking you to do busywork. If you'd been watching the numbers the way I'm about to ask you to, I believe you would have caught this morning's "zero successes out of 800" faster than I did — you wouldn't have been distracted trying to fix anything, you'd just have been watching the number. That's a real, necessary job on a project this size, and it's yours starting now. Exact steps, exact commands — I'm not going to make you guess at anything.

1. CHECK PROGRESS. Log into Bluehive the way you normally do, then run this one line exactly as written (activates the right Python environment, prints a live status report):

```
source /scratch/tolugboj_lab/softwares/anaconda/anaconda3/2021.05/etc/profile.d/conda.sh && conda activate instaseis && python3 /scratch/tolugboj_lab/wavenet_ncf_framework/production/master.py progress --root /scratch/tolugboj_lab/wavenet_ncf_canary
```

([wavenet_ncf_canary](#) is the 60-station test batch running right now. Once I scale up to the real full run, I'll give you the new path — same command, different `--root`.)

That prints a progress bar, GB downloaded/packaged, and a line "with real data: N".

Watch for:

- "with real data" at 0, or staying suspiciously low while "orchestrator tasks reported" keeps climbing — that's exactly the red flag I missed this morning. Don't wait for me to notice, message me right away.
- Run the same command again a few hours later. If the numbers haven't moved, something is stuck.

2. IF SOMETHING LOOKS WRONG, CHECK THE ACTUAL ERRORS. This one line tells you how many log files mention an error (a repeating identical error means a real bug, not one unlucky station):

```
grep -l "Error" /scratch/tolugboj_lab/wavenet_ncf_canary/logs/*.err | wc -l
```

To actually read a few results, plain and readable, run:

```
for f in $(ls /scratch/tolugboj_lab/wavenet_ncf_canary/results/ | head -5); do echo "--- $f ---"; cat /scratch/tolugboj_lab/wavenet_ncf_canary/results/$f; echo; done
```

Don't worry about the exact fields, just read it like a sentence. Same error text in more than one file is the pattern to report.

3. CHECK THE SLURM QUEUE ITSELF:

```
squeue -A tolugboj_lab -o '%.12i %.20j %.10P %.8T %.10M'
```

Look for anything stuck "PENDING" a long time, or a batch of tasks all finishing within seconds of each other (usually means they're all failing fast, not

succeeding fast).

4. ONCE THERE'S REAL PACKAGED DATA, SPOT-CHECK A FEW STATIONS. Same kind of check you're already doing for Task B below, applied to live output instead of a fixed reference file. I'll walk you through the exact load-and-check commands once there's real merged data to look at — steps 1-3 above are what I need right now.

5. WHAT TO TELL ME: which station(s) (network.station, e.g. "II.NNA"), the exact error text if any, and roughly how many other stations show the same pattern. "Stations around idx 500-600 are all failing with the same error text" is something I can act on immediately; "some downloads look broken" isn't. And just as important: "checked at 2pm and 3pm, looks healthy, no repeating errors" is a completely valid, useful report. Silence is the only wrong answer.

Starting today: run step 1 against the canary batch every hour or two and tell me what you see, good or bad. Once I scale to the real 2,000-station run (I'll message the team), do the same against that, for at least the first several hours.

TASK B — Reproduce the GT.BOSA correctness check yourself, independently

Don't just read my numbers in End-notes A and move on. To make this easy — no [atos](#) access needed — I staged exactly what you need on repovibranium (same access you already have per the team onboarding memo):

/volume1/web/Wavenet/NCF_Verification/

gt_bosa_days_057-060_raw.tar -- raw SEED, GT.BOSA, 1993 days 057-060 (39 MB)

bosa_reference_057-060.tar -- matching ADAMA reference SAC (2.7 MB)

README.md -- exact steps

Our code: src/wavenet_pipeline/00_waveform_acquisition/rover_download/

preprocess_existing_archive.py (produces the packaged output) and

compare_against_reference.py (the comparison approach — hardcoded to

this exact run, adapt the paths/dates, it's a worked example not a CLI

tool).

(Repovibranium's [scp/sftp](#) doesn't work either, same Synology quirk as [atos](#) — use [ssh repovibranium "cat file" > local_file](#), or just browse/download via the web share if that's easier.)

Get your own correlation/amplitude numbers. If they don't match mine, that discrepancy matters more than either of us assuming the other is right.

TASK C — XD.MTAN / XD.RUNG, channel BHZ (paper-motivated follow-up)

These two stations are the ones from our paper — a real, scientifically-motivated pair, not an arbitrary one. You do NOT need [atos](#) access for this one; I already confirmed both are fully available through the

same public FDSN service our MassDownloader pipeline uses:

XD.MTAN: -7.9073, 33.3203, deployed 1994-05-25 to 1994-10-30

XD.RUNG: -6.9372, 33.5180, deployed 1994-05-25 to 1995-05-16

~110 km apart, overlapping window 1994-05-25 to 1994-10-30 (~5 months) — real

geometry, not just structurally present.

Use the existing, already-proven MassDownloader pattern (`mdl_bluehive_quicktest.py`) pointed at `network="XD"`, `station="MTAN"` and `"RUNG"`, `channel="BH?"` (this 1994 deployment predates the LH-preference optimization's relevance — BHZ specifically is what's wanted here), over the overlapping window above. This is a download-based test, not Task B's download-free archive-replay style — expect real download/ preprocess time per End-notes E's numbers.

Once both stations are packaged and loaded (`build_master_h5.py` + `load_master_h5.py`, same as the rest of this pipeline): do the results look sane heading into NCF cross-correlation? Confirm both series cover the same real calendar range, then try ObsPy's own `obspy.signal.cross_correlation.correlate` on the two loaded arrays as a basic smoke test — full NCF (Stage 5) doesn't exist in this pipeline yet, so this is a readiness check, not a claim Stage 5 is built.

One more thing worth checking yourself, not something I've verified: there may already be processed SAC data for XD.RUNG on terravibranium under `/RAID6/bluehiveBackup/Prj12_AfrTxRF/2_Data/SAC/XD/RUNG/` — I haven't confirmed whether those are continuous-day traces (comparable to our output) or event-cut receiver- function segments (different purpose, not directly comparable) — worth a quick look before assuming it's a second trusted reference the way GT.BOSA was.

HOW TO REPORT BACK (all three tasks)

- Update `docs/ncf_pipeline_stages/PROGRESS.md` and the relevant stage doc's Hardware Tier Log the same way this session's work is already tracked — same convention, just your name on the tier row instead of mine.

- "I reproduced it, same numbers, no concerns" is a completely valid report.

- If your numbers differ from mine, or anything doesn't look right for a reason I haven't anticipated, say so plainly and immediately, even if you're not sure it's really a problem. I would much rather you flag ten things that turn out fine than stay quiet about the one that doesn't.

END-NOTES (context and evidence behind the TL;DR — for follow-up, not required reading)

A. WHY OBSPY + BLUEHIVE, AND THE CORRECTNESS EVIDENCE

I picked ObsPy's MassDownloader over ROVER (ROVER's high-level commands depend on a now-retired FDSN service; MassDownloader doesn't, and it fetches StationXML for free) and Bluehive as the execution home (its [urseismo](#) partition — 120 cores, exclusive to us, confirmed zero queue wait even at 20-way concurrency).

The gap that mattered most — is our instrument-response removal actually correct, not just non-crashing — is closed with a real, independent check, not a plausibility argument. I found ADAMA's own already-processed output for GT.BOSA (a real station, part of the GT network, on terravibranium: /RAID6/bluehiveBackup/Prj5_HarmonicRFTraces/2_Data/preprocessed_data/DataGT/BOSA/) — SAC files independently produced by a different pipeline than ours. I ran our own preprocessing against the same raw day (1993-058, pulled from a real archive on [atos](#)) and compared numerically:

min/max agree to 4-5 significant figures

amplitude std ratio: 0.9998

correlation coefficient: 0.9523

Two independently-built pipelines, same raw instrument response, near-identical displacement output. That's the number Task B asks you to reproduce yourself.

Also real, not projected: a 27-year single-station stress test (II.EFI, 1,875 real days, zero download cost since the raw data already existed on Bluehive) surfaced two things a short toy test never would have:

- Non-standard channel orientations are real (one channel's true azimuth was 343°, not aligned to north at all) — confirms the azimuth/dip metadata we store per channel is load-bearing, not over-engineering.
- Real instrument sample rates aren't exactly nominal (0.999999713897705 Hz, not 1.0) — over 27 years that compounds to an estimated ~243 seconds of time drift if uncorrected. Flagged in hdf5_schema.md, not yet resolved — worth remembering for anything doing precise multi-decade alignment.

B. THE FULL-HISTORY DECISION AND THE PRODUCTION FRAMEWORK

Decision (2026-09-24): download all available data for every station, not a connectivity-gated subset — "the goal is to get all the data." Real numbers from the key index once this was decided: 78.9 TB total, spanning from a few dozen days up to 56 years for the longest-running stations (median far smaller — most of that volume and time sits in a long tail of ~40 outlier stations).

I built a production framework to run this at scale — a master/orchestrator/logger/ inspector split, living at [src/wavenet_pipeline/00_waveform_acquisition/production/](#) on the [add-september-ncf-pipeline](#) branch. One SLURM job per station downloads and packages it, one process merges finished stations into a shared file, one process double-checks each merge and only then deletes the raw download, and one script drives all of that. Full design detail: [docs/ncf_pipeline_stages/PROGRESS.md](#).

While building this I hit and fixed four real infrastructure bugs (a conda environment conflict, a race condition between two copies of the same monitoring process, an early-exit bug, and a SLURM job-numbering limit) — all verified on a small 15-station test before I touched the full 2,000. The framework itself was solid before the incident in C below; what went wrong next was a different kind of

problem.

C. THE INCIDENT: WHAT HAPPENED AT FULL SCALE, AND THE FIX

I launched all 2,000 stations. After about 800 finished, zero — not some, zero — had actually succeeded. The cause: ObsPy itself has a bug that triggers when asking for a station's full history spanning many decades — sometimes it crashes outright, sometimes it fails silently and reports "no data here" when the station actually has plenty. I proved this by re-running two of the broken stations by hand after my fix; both came back with real, correct data. Fixed by requesting one year at a time instead of all 56 years in one request. A small 60-station canary batch (the one Task A asks you to watch) is confirming the fix holds before I trust it with the full 2,000 again.

Why my own pre-launch testing didn't catch this: every test before this decision used a narrow one-week window, so none of it could have ever hit a bug that only shows up on a multi-decade request. What I checked right before launching — watching one station start downloading correctly — proved the mechanism worked, not that it would keep working across hundreds of real stations and providers. That gap (watching for "does it crash" instead of "does the success rate across many stations look right") is exactly why Task A exists now — it's a distinct job from building the pipeline, and I was doing it badly single-handedly because my attention was on the fix, not the watch.

D. THE SECOND ROBUSTNESS GAP, AND HOW I'M TESTING THE FIX

Separately, I asked myself: if a station's job dies mid-run (preempted, times out, crashes) what do we actually lose? Two answers.

Expected: raw downloaded files are safe. Confirmed directly, not assumed — while the canary run was in progress, I checked disk and found stations that hadn't finished yet already had thousands of real files sitting there (G.SSB had 1,896, II.NIL had 12,576, as one snapshot). Those are just files in a folder; killing the job doesn't touch them.

Not expected: the step after download — turning raw data into our packaged format — currently has to happen in one uninterrupted pass over a station's *entire* history, all in memory, before any of it is saved. If that step is interrupted, all of it is lost and restarts from scratch, even though the raw files it needed survived. Worse, for our biggest stations this step likely needs far more memory than we're currently giving it, meaning it may simply crash on its own regardless of any external interruption.

I'm testing before building: downloading one full year of our single largest station and measuring, directly, how much memory the existing processing step actually uses. If a year fits comfortably even for our worst-case station, checkpointing progress once per year (save what's done, don't redo it if the job dies) is both safe and enough. If a year is still too much, that tells me I need a finer checkpoint — by month or by day — and the test will make that obvious instead of me guessing wrong and finding out in production again. I'll circulate the real numbers once this test finishes; I'm not changing how the pipeline processes data until I have them.

Also fixed along the way: a narrow race where checking progress at the exact wrong moment could catch a station's result file mid-save and crash the progress command. The save is now atomic (a check only ever sees a complete file, never a half-written one), and the progress command is more defensive on top of that. If you or I ever see two progress checks minutes apart show different numbers, the far more likely explanation is simply that the run genuinely moved in between — expected behavior on something

actively in progress, not a bug.

E. OBSPY / BLUEHIVE TIME, STORAGE & SCALING NUMBERS (feeds the AWS cost report)

All confirmed by direct testing (single-station runs, a 20-task SLURM array, and the 27-year stress test above) — not projections:

Preprocessing (the real bottleneck, not download):

~5-6 s/channel-day when a channel is already at our 1 Hz target rate (LH?)

~9-16 s/channel-day when only BH? exists and needs decimating from a much

higher native rate (real station-to-station variance seen:

9.1s for one station, 16.1s for another, both legitimate)

Storage: ~300-410 KB/channel-day (real range across different stations/content, not a single fixed number) — essentially flat regardless of channel type or concurrency level, since final output is always decimated to 1 Hz. gzip-4 compression only saves ~7% on this data (it's high-entropy after filtering) — if storage ever becomes a real concern at full network scale, the lever is int16 quantization or a coarser rate, not compression tuning.

Scaling: linear from 1-way to 20-way concurrency on urseismo — no measurable slowdown from provider rate-limiting or shared-filesystem contention at that scale. 20 tasks all entered RUNNING immediately, zero queue wait.

The real-archive stress test also reinforces why raw SEED must be discarded after packaging, not kept: /RAID6/Prj_terraSeis/output/ on terravibranium is 2.5 TB of kept raw SEED for a regional project; PrjXX_SAmericaNoise's own raw archive on Bluehive scratch is 11 TB. Real numbers, not a hypothetical risk.

F. SCALE PREVIEW TABLE (illustrative, predates the full-history decision in B)

This table used the pre-decision assumptions (bounded windows) to get everyone's head around scale before the full-history choice was made — keeping it here as a reference point, not the current plan.

Assumptions: ~5 channels/station, ~10 s/channel-day preprocessing, ~350 KB/channel-day storage, 2,000 stations fixed.

Scenario	Channel-days	Serial time	Time on urseismo (120-way parallel)	Storage
1 week/station	70,000	~8.1 days	~1.6 hours	~24.5 GB
1 year/station	3,650,000	~422 days	~3.5 days	~1.28 TB
Full multi-decade history (illustrative, using II.EFI's 1,875-day/27-year span)	18,750,000	~5.9 years	~18 days	~6.6 TB

Parallelization is what makes any of this tractable at all — the difference between "5.9 years" and "18 days" is entirely [urseismo](#)'s 120 cores, confirmed linear at least up to 20-way concurrency (not yet confirmed all the way to 120-way — a real assumption in this table, not proven). Scope choice matters enormously: full-history is a ~270x larger problem than a 1-week bound, in both time and storage — exactly why B's actual decision to go full-history is the bigger, real commitment it is.

Memory (from the 27-year II.EFI test, real number not an assumption): 551 MB RSS checked ~1 hour in, confirming the day-by-day processing design (never holds more than one day's data across all channels at once) keeps memory low regardless of history length — this is the SAME design principle D's checkpointing fix is meant to bring to the production orchestrator, which currently does NOT follow it.

— Tolu